

Guia Prático de Migração: De JavaScript para TypeScript

Neste material, vamos transformar o nosso projeto JavaScript puro em um projeto TypeScript moderno, seguro e com ambiente de desenvolvimento ágil (Hot Reload).

A transição será feita passo a passo. O objetivo não é apenas decorar comandos, mas entender **por que** cada ferramenta existe no ecossistema do TypeScript.

Introdução: Por que estamos fazendo isso?

Lembre-se da regra de ouro: **O Node.js e os Navegadores não entendem TypeScript**. Eles só falam JavaScript.

O TypeScript é uma ferramenta de "tempo de desenvolvimento". Ele atua como um corretor ortográfico super rigoroso no seu VS Code. Quando terminamos de programar, precisamos de uma ferramenta para "traduzir" (compilar) esse código seguro de volta para o JavaScript clássico, para que o Node.js consiga executá-lo.

Nossa missão se divide em três fases:

1. **Ensinar o projeto a ler TypeScript.**
2. **Criar um ambiente de testes rápido (Hot Reload).**
3. **Gerar a versão final do código para Produção.**

Fase 1: Preparando o Terreno (Instalações)

Abra o terminal na pasta raiz do seu projeto (onde está o seu `package.json`). Vamos instalar as ferramentas necessárias.

Execute o comando:

```
npm install -D typescript @types/node tsx
```

O que acabamos de instalar?

- **typescript** : O cérebro da operação. O compilador que entende a linguagem e verifica os erros.
- **@types/node** : O TypeScript não conhece os comandos nativos do Node (como `fs`, `path`, `console`). Este pacote funciona como um "dicionário" que ensina ao TS as regras do Node.js.
- **tsx** : Uma ferramenta moderna incrível. Antigamente usávamos a combinação de `nodemon` com `ts-node`. O `tsx` faz o trabalho dos dois: ele roda arquivos TypeScript diretamente no terminal e reinicia o servidor sozinho quando você salva o arquivo (Hot Reload).

(Nota: O `-D` significa que estamos instalando como dependência de desenvolvimento, pois o servidor de produção final não precisará delas).

Fase 2: O Manual de Regras (`tsconfig.json`)

Agora que o TypeScript está instalado, precisamos criar o arquivo de configuração dele. Esse arquivo dirá ao compilador quão rigoroso ele deve ser e onde ele deve guardar os arquivos traduzidos.

No terminal, rode:

```
npx tsc --init
```

Isso vai gerar um arquivo chamado `tsconfig.json` gigante, cheio de comentários. Para o nosso projeto, você pode apagar tudo o que está lá dentro e substituir por esta configuração base limpa:

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "CommonJS",
    "rootDir": "./src",
    "outDir": "./dist",
    "esModuleInterop": true,
    "forceConsistentCasingInFileNames": true,
    "strict": true,
    "skipLibCheck": true
  },
  "include": ["src/**/*"]
}
```

Traduzindo as regras principais:

- **rootDir ("Pasta Raiz")**: Avisa ao TypeScript que todo o nosso código fonte estará dentro de uma pasta chamada `src`.
- **outDir ("Pasta de Saída")**: Diz ao compilador: "Quando você traduzir o `.ts` para `.js`, não suje a minha pasta de trabalho. Jogue tudo numa pasta separada chamada `dist` (Distribution)".
- **strict: true**: Liga a proteção máxima. O compilador não vai deixar passar variáveis sem tipo ou possíveis valores "nulos" perigosos.

Fase 3: Organizando a Casa (Migração Física)

Se você não tem uma pasta `src`, crie uma agora e mova todo o seu código JavaScript (`.js`) para dentro dela.

A regra de ouro da migração contínua: **Renomeie um arquivo de cada vez.**

1. Pegue o seu arquivo principal (ex: `index.js` ou `server.js`).
2. Renomeie a extensão de `.js` para `.ts`.
3. Seu VS Code provavelmente vai começar a mostrar alguns sublinhados vermelhos. Não se desespere. Isso é o TypeScript dizendo: *"Ei, você não me disse qual é o tipo dessa variável"*.
4. Adicione os tipos básicos (ex: `nome: string, idade: number`) para acalmar o compilador.

Fase 4: O Ambiente de Desenvolvimento (Hot Reload)

Durante o desenvolvimento, nós não queremos ficar rodando comandos de compilação toda hora que salvarmos o arquivo. Queremos agilidade. É aqui que o `tsx` brilha.

Abra o seu arquivo `package.json` e localize a sessão `"scripts"`. Vamos adicionar os nossos comandos de atalho:

```
"scripts": {
  "dev": "tsx watch src/index.ts",
  "build": "tsc",
  "start": "node dist/index.js"
}
```

O que faz o comando `dev`?

Quando você rodar `npm run dev` no terminal, o `tsx` entra em ação. A palavra `watch` significa "vigiar". Ele vai ligar o seu servidor e ficar observando a pasta `src`. Sempre que você apertar `Ctrl + S` para salvar um arquivo, ele reinicia o servidor instantaneamente. **Esse é o seu Hot Reload.**

Fase 5: A Mágica da Compilação (Produção)

O seu projeto está pronto, tipado e rodando perfeitamente no ambiente `dev`. Agora precisamos preparar isso para colocar na internet (produção).

Lembre-se: Servidores de nuvem (como AWS, Render, Heroku) são configurados para rodar Node.js com JavaScript puro, por ser mais rápido e leve.

No terminal, rode o comando:

```
npm run build
```

O que acabou de acontecer? O TypeScript (`tsc`) pegou todos os seus arquivos `.ts` de dentro da pasta `src`, removeu todas as tipagens (sim, os tipos somem!), verificou se não havia nenhum erro lógico, e gerou arquivos JavaScript puros (`.js`) limpíssimos dentro da pasta `dist`.

Agora, para simular o ambiente de produção rodando o código final, você usa o comando:

```
npm run start
```

Isso fará o Node.js olhar diretamente para a pasta `dist` e executar o código compilado.

Resumo do Ciclo de Vida do Projeto

Para garantir que o fluxo de trabalho ficou claro, o dia a dia do programador Back-End moderno funciona assim:

1. **De Manhã (Desenvolvimento)**: Você abre o projeto e roda `npm run dev`. Trabalha o dia inteiro escrevendo em arquivos `.ts` na pasta `src`.
2. **De Tarde (Compilação)**: O código ficou pronto e testado. Você roda `npm run build`. A pasta `dist` é criada/atualizada com os arquivos `.js`.
3. **De Noite (Deploy)**: Você manda o projeto para o servidor, que roda o `npm run start` para manter a aplicação viva usando o JavaScript traduzido.

Missão Prática para os Alunos:

1. Instalem as 3 dependências mostradas na Fase 1.
2. Criem o arquivo `tsconfig.json` e a pasta `src`.
3. Crie um arquivo `src/index.ts` com um simples `console.log("Olá TypeScript!");`.
4. Configurem os scripts no `package.json` e executem `npm run dev`. Modifiquem o texto e vejam o terminal atualizar sozinho.
5. Em seguida, parem o servidor e rodem `npm run build`. Abram a pasta `dist` e vejam como o código foi traduzido!